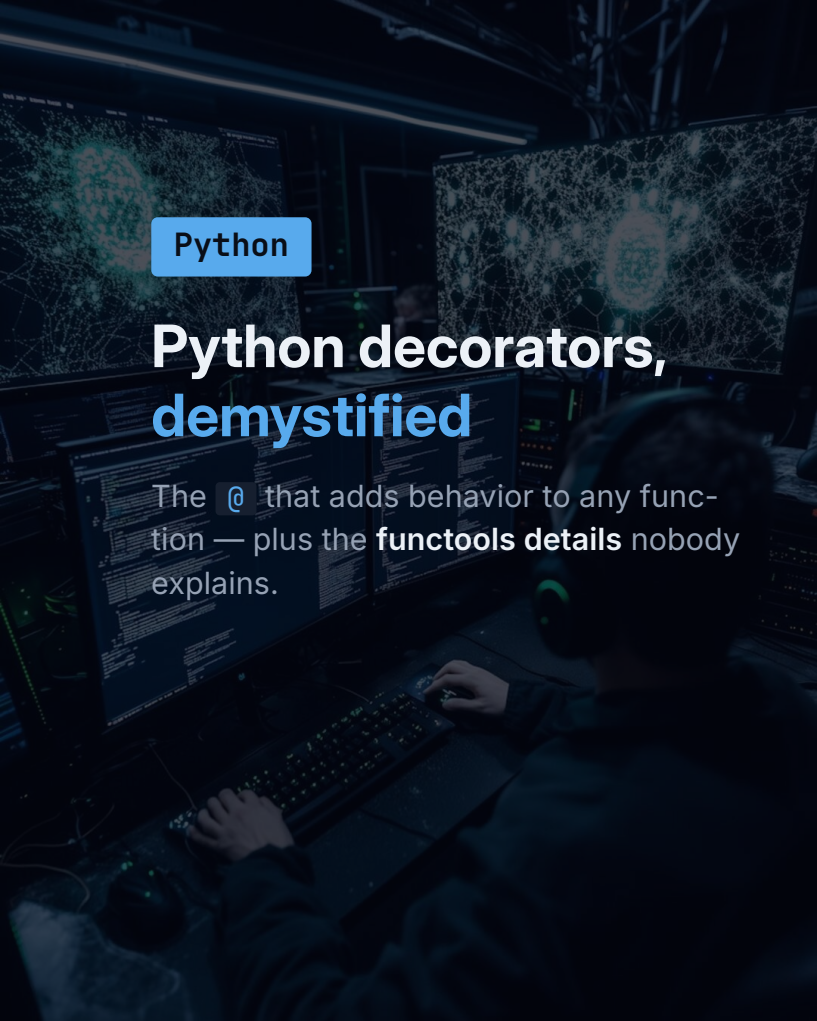
Python
Python decorators, demystified
The @ that adds behavior to any function — plus the functools details nobody explains.

-- before the syntax

A decorator wraps a function

It takes a function and returns a new one that runs extra logic around it — logging, timing, caching, access checks — while the original code stays untouched.

- The `@` is just sugar: `@log` above `f` means `f = log(f)`.
- You've used them already: `@property` and Flask's `@app.route`.

> WHY IT MATTERS
>
> It's how frameworks add behavior declaratively — one line above a function instead of ten inside it.

-- 01 · the pattern

Take a function, return a new one

pattern.py

```python
def log(fn):
    def wrapper(*args, **kwargs):
        print(f"-> {fn.__name__}")
        return fn(*args, **kwargs)
    return wrapper
```

GOTCHA

`@log` above a function is nothing magic — it's just `f = log(f)` run for you.

-- 02 · keep the identity

Always add @wraps

wraps.py

```python
from functools import wraps
def log(fn):
    @wraps(fn)
    def wrapper(*a, **k):
        return fn(*a, **k)
    return wrapper
```

GOTCHA

Without it, the name becomes `"wrapper"`, the docstring vanishes, and debugging/introspection break.

-- 03 · decorators that take args

Add arguments? Three layers

args.py

```
def retry(n):
    def deco(fn):
        def wrapper(*a, **k):
            ...
        return wrapper
    return deco
```

GOTCHA

Three layers: the **arg**, the **function**, the **wrapper**. `@retry(3)` calls `retry(3)` first, then decorates.

-- 04 · stacking

Stacked decorators run bottom-up

order.py

```
@bold
@italic
def greet(): ...
# greet = bold(italic(greet))
```

GOTCHA

The decorator **nearest the function** wraps first. Flip the order and you flip the output.

-- 05 · don't reinvent it

The stdlib ships decorators

cache.py

```python
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n-1
    ) + fib(n-2)
```

PRO TIP

Before writing your own, reach for `lru_cache`, `cached_property`, `singledispatch`, `wraps`.

-- 06 · methods as attributes

@property hides the parentheses

property.py

```python
class Circle:
    def __init__(self, r):
        self.r = r
    @property
    def area(self):
        return 3.14159 * self.r**2
```

GOTCHA

It reads like data — `c.area`, not `c.area()`. Add `@area.setter` to guard writes.

-- 07 · on classes

classmethod vs staticmethod

methods.py

```
class User:
    @classmethod
    def from_json(cls, d): return
    cls(**d)
    @staticmethod
    def valid(e): return "@" in e
```

GOTCHA

`classmethod` gets `cls` — great for alternate constructors. `staticmethod` gets nothing.

-- what trips people up

The traps that cost you hours

- No `@wraps` — broken names, lost doc-strings, confused debuggers.
- Decorator body runs **once at definition**; the wrapper runs **every call**.
- Shared mutable state in the closure leaks across calls.
- Need per-instance state? Use a **class-based** decorator.

> PRO TIP
>
> If it holds state or config, a class with `__call__` is clearer than three nested functions.

-- go deeper

Now decorate with intent

Full write-up + runnable code for every example — scan and clone it.

Alejandro Sánchez Yalí
Software Developer | AI & Blockchain Enthusiast
www.asanchezyali.com

the full guide

code + deep-dive on GitHub